Whitepaper

Serenity BDD vs Playwright vs Cucumber-only

A Practical Comparison for SDET Teams

Author: Kavita Jadhav

Organization: k11 software Solutions LLC

Published Date: April 24, 2026

Serenity BDD vs Playwright vs Cucumber-only for SDET teams

Executive summary

For most **greenfield web UI automation** work, **Playwright** is the strongest default. Its official stack gives you browser automation, a built-in test runner, isolation, parallelization, reporters, tracing, and API testing in one package; its community footprint is also much larger than Serenity's current footprint. [1]

For **Java-first teams** that care deeply about **rich reporting, requirements traceability, and living documentation**, **Serenity BDD** remains the strongest choice. The most important 2026 nuance is that Serenity is **not only a Selenium-era framework** anymore: its current docs include official **Serenity Playwright** modules and **Cucumber integration with Playwright**, so a JVM team can combine **Playwright's browser engine** with **Serenity's reporting and Screenplay conventions**. [2]

For teams evaluating **Cucumber-only**, the key decision is whether **Gherkin is a real product/acceptance contract** or just a familiar habit. Official Cucumber docs are explicit that Cucumber is **not** a browser automation tool and **not** an API automation tool; it is the BDD/specification layer, and you must choose the automation engine, assertion library, reporting model, and often DI/state conventions yourself. That makes it flexible and widely portable, but less turnkey than either Serenity or Playwright. [3]

The most practical shorthand is this: **choose Playwright for modern browser automation and fast feedback; choose Serenity for Java teams that want stakeholder-grade reports and living documentation; choose Cucumber-only only when Gherkin itself is strategically valuable.** [4]

What these stacks actually are

A fair comparison starts by recognizing that these three options are **not symmetric**. **Playwright** is a full automation-and-runner stack. **Serenity BDD** is an opinionated JVM test architecture and reporting layer that can sit on top of different execution approaches, including Selenium/WebDriver and now official Serenity Playwright. **Cucumber-only** is a BDD execution/specification layer built around Gherkin and step definitions, not a browser engine. [5]

Serenity's official material makes its design intent very clear: it emphasizes **Step Libraries**, **Page Objects**, and especially the **Screenplay Pattern**, plus reports that document not only results but also **what requirements and features have been tested**.

Its current docs also present two Serenity Playwright approaches—traditional Page Objects and modern Screenplay—alongside Cucumber integration. [6]

Playwright's official docs position it as a modern end-to-end framework with **bundled runner, assertions, isolation, parallelization, reporters, tracing, cross-browser support, and API testing**. Playwright is available in multiple languages, but the docs explicitly note that **testing ecosystem integration differs by language**, with the richest out-of-the-box experience in **Node.js/TypeScript** via Playwright Test. [7]

Cucumber's official docs position it as a tool for **plain-language executable specifications**. It is intentionally broad across languages and implementations, but that breadth comes with composition work: for browser automation, Cucumber points you to tools like Selenium WebDriver; for API automation, it points you to tools like RestAssured, PactumJS, or Ruby HTTP clients. [8]

Attribute comparison

The table below treats **Serenity BDD** as the full Serenity stack, **Playwright** as Playwright Test / official Playwright stack, and **Cucumber-only** as Cucumber without an opinionated surrounding framework.

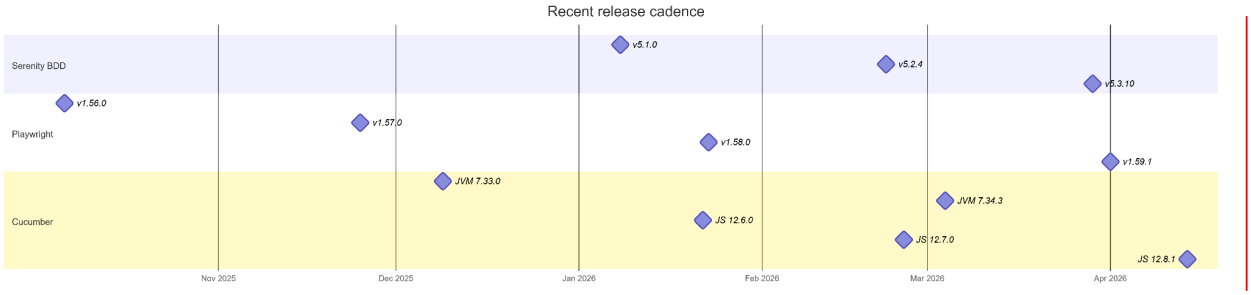
Attribute	Serenity BDD	Playwright	Cucumber-only
Language and platform support	**JVM-first and Java-centric** in current docs and tutorials; official docs now include Serenity Playwright on the JVM.	**Most integrated in Node.js/TypeScript**, with official support for Python, Java, and .NET; docs note ecosystem integration differs by language.	**Broadest overall language surface** because Cucumber ships multiple official and semi-official implementations across mainstream languages.
Test style	Supports **BDD with Cucumber**, **traditional Page Objects/Step Libraries**, and **Screenplay**.	Primarily **scripted/test-runner-first** with fixtures, projects, annotations, and code-based tests.	**Gherkin + step definitions** first.
Reporting and living documentation	Strongest of the three for **living documentation**, requirement hierarchy, feature coverage, screenshots, and stakeholder-facing reports.	Strong for engineers: **HTML report**, traces, screenshots, videos, merged reports; weaker than Serenity for requirements-oriented living docs.	Built-in and extensible **formatters/reporters**, plus third-party plugins and Cucumber Reports, but not the same out-of-the-box requirements view as Serenity.
Browser automation capabilities	Depends on chosen engine: official docs cover **Selenium/WebDriver** and **Serenity Playwright**.	First-class **Chromium, Firefox, WebKit**, branded browsers, device emulation, network interception, traces, and resilient locators.	**None built in**; you must pair Cucumber with a browser engine such as Selenium.
API testing support	Native story via **RestAssured** and Serenity Screenplay REST.	Built-in **APIRequestContext**, including cookie-sharing with browser contexts.	**External by design**; official docs point to tools like RestAssured, PactumJS,

Attribute	Serenity BDD	Playwright	Cucumber-only
			HTTParty, and others.
CI and CD integration	Official docs cover **GitHub Actions, Jenkins, and GitLab CI**; Serenity reports are meant to be published in CI.	Official docs cover **any CI provider**, generated **GitHub Actions** setup, artifacts, sharding, and Docker images.	CI works well, but more through **formatter output and CI-server integration** than a single unified stack.
Test stability and flakiness handling	Stability is partly **engine-dependent**. Serenity adds wrappers, Page Objects, screenshots, page-source capture, and reports for diagnosis; if you use Serenity Playwright, you also inherit Playwright's auto-waiting.	Strongest out-of-the-box: **actionability checks**, **auto-retrying assertions**, **fresh browser contexts**, **retries**, and rich diagnostics.	Some implementation help exists—fresh **World** / scenario isolation and, in Cucumber-JS, targeted **retry**—but browser flakiness handling mostly comes from the underlying engine and test design.
Learning curve and community activity	Usually steeper because you learn **Serenity conventions** plus often **Screenplay**; community is active but smaller and more niche.	Usually fastest to justify for modern web teams; larger current community signal across GitHub, docs, Stack Overflow, and Reddit.	Conceptually simple if your org already speaks BDD, but maintenance overhead rises because you assemble the rest of the stack yourself. Community is broad, but fragmented by language and implementation.

Attribute	Serenity BDD	Playwright	Cucumber-only
Maintenance and release cadence	Active 5.x line in 2026; recent releases landed in January, February, and March 2026.	Very active release train; official releases show continuing 1.56–1.59 activity through April 2026.	Active, but cadence is split across implementations such as **Cucumber-JVM** and **Cucumber-JS** rather than one monolith.
Ecosystem and tools	Broad JVM ecosystem: Selenium/WebDriver, Playwright modules, REST, Screenplay, Appium, BrowserStack, Jira, Maven plugin, extra reports.	Broad modern tooling: VS Code extension, Docker images, CLI, MCP, HTML reporter, trace viewer, annotations, sharding.	Broad plugin/tooling surface: editors, IDE support, formatters, reports, DI modules, but more self-assembly.
Performance and parallelism	Parallel execution is supported, especially with **JUnit 5 + Cucumber 7**, but overall runtime still depends heavily on the chosen engine.	Strong default: **parallel by default**, worker processes, and fresh browser contexts with low overhead.	Parallelism exists in both **Cucumber-JVM** and **Cucumber-JS**, but setup and behavior vary by implementation.
Typical teams and use cases	Best fit for **Java enterprise SDETs**, regulated environments, BDD-heavy teams, and organizations that want reports useful to QA, product, and stakeholders.	Best fit for **modern web product teams**, greenfield suites, CI-first teams, and anyone optimizing for fast browser feedback loops.	Best fit when **Gherkin is the product contract** across multiple teams or languages and you intentionally want to choose the engine separately.

All three options show clear signs of active maintenance in 2026, but the shape of that activity differs. **Playwright** shows the largest public footprint here, with **87.1k GitHub stars**, **160 releases**, and a latest GitHub release of **v1.59.1** on **April 1, 2026**. **Serenity BDD** is much smaller publicly, but still active, with **5.3.10** published on **March 29, 2026** in Maven and recent 5.1/5.2/5.3 release lines. **Cucumber** is best read as an ecosystem: **Cucumber-JS** shows **5.3k stars** and **v12.8.1** on **April 14, 2026**, while **Cucumber-JVM** shows **2.8k stars** and **v7.34.3** on **March 4, 2026**. [21]

The timeline below uses the official release pages and registries cited above. [22]



Pros, cons, and team fit

Stack	Biggest pros	Biggest tradeoffs	Best default when
Serenity BDD	Richest **living documentation** and stakeholder reporting; strong **Screenplay** and **BDD/JVM architecture**; can now pair with **Playwright** as well as Selenium; solid REST story.	Smaller public community footprint; steeper learning curve; strongest if your team is already comfortable in the JVM.	You are a **Java-heavy SDET team** and reporting/readability matters almost as much as execution.
Playwright	Best overall balance of **modern browser automation**, **stability tools**, **parallelism**, **CI friendliness**, and **fast setup**.	Out-of-the-box reporting is more engineering-centric than business/requirements-centric; full “best experience” is strongest in Node.js/TypeScript.	You want **greenfield browser automation** with the least extra assembly.

Stack	Biggest pros	Biggest tradeoffs	Best default when
Cucumber-only	Excellent when **Gherkin is the shared contract**; broad language reach; maximal flexibility to pair with different UI/API stacks.	Least batteries-included; browser/API/assertion/reporting choices are external; can create step/glue maintenance overhead if BDD is not culturally real.	Your org truly wants **BDD as collaboration language**, not just tagged tests.

The most important practical takeaway for experienced SDETs is that **Serenity and Playwright are no longer clean substitutes**. If your team wants **Playwright-class browser automation** but also wants **Serenity-grade reporting and living documentation**, the official Serenity Playwright path gives you that combination on the JVM. In other words, the real architecture conversation is often **"Playwright alone" vs "Playwright inside Serenity,"** not simply **"Serenity vs Playwright."** [26]

A second practical takeaway is that **Cucumber-only is often orthogonal to the other two**. Cucumber answers **"Do we want Gherkin and executable specifications?"** Playwright answers **"What browser/API automation stack do we want?"** Serenity answers **"How opinionated should our architecture, reporting, and stakeholder visibility be?"** That is why "Cucumber-only" often feels simultaneously attractive and incomplete. [27]

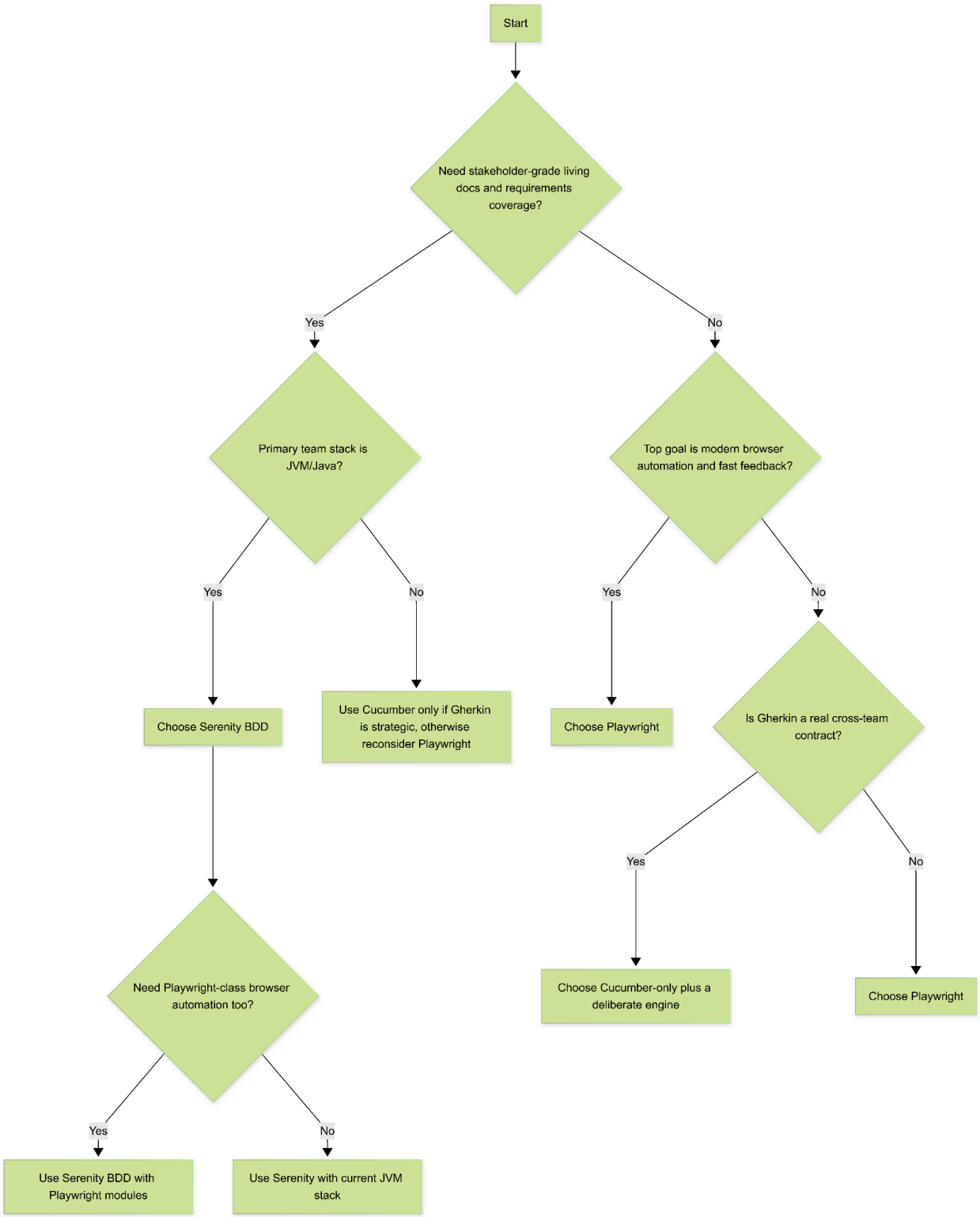
Decision matrix

The recommendations below are interpretive, but each one is grounded in the documented capabilities above.

Team priority	Recommended choice	Why this is the pragmatic fit
Java ecosystem first	**Serenity BDD**	Serenity's current docs, tutorials, and surrounding modules are JVM-first; it integrates naturally with JUnit/Cucumber and gives a stronger Java-native architecture than Playwright alone.
Java ecosystem + modern browser automation	**Serenity BDD with Playwright modules**	This preserves Playwright's engine advantages while adding Serenity's reporting, screenshots, step model, and optional Cucumber/Screenplay architecture.

Team priority	Recommended choice	Why this is the pragmatic fit
Rich reports and living documentation	**Serenity BDD**	Serenity is the only one here with a first-class requirements view, functional coverage, and explicit living-documentation model.
Modern browser automation	**Playwright**	Cross-browser support, modern locators, auto-waiting, trace viewer, isolation, mobile emulation, and official CI patterns are all first-class.
Fast feedback and parallel CI	**Playwright**	Playwright runs in parallel by default, uses isolated browser contexts, supports sharding, and includes strong CI reporting/trace workflows.
Strong BDD emphasis	**Serenity BDD** if you are JVM-heavy; **Cucumber-only** if Gherkin must stay tool-agnostic	Serenity gives BDD plus powerful reporting; Cucumber-only is better when Gherkin is the real center of gravity across mixed stacks.
Greenfield UI suite	**Playwright** by default	It is the most complete out-of-the-box stack for new browser suites, especially in Node.js/TypeScript.
Legacy Cucumber/Selenium suite	**Serenity BDD** for structured evolution; **Cucumber-only** if you only need to preserve Gherkin	Serenity offers a more incremental upgrade path for reporting and parallelism without forcing a full conceptual reset; Cucumber-only keeps the BDD layer but leaves the rest unchanged.
Polyglot org or tool flexibility	**Cucumber-only**	Cucumber spans more platforms and lets you choose engine and assertions per ecosystem.

The flowchart below condenses the same logic into a quick “pick a stack” path. It is based on the documented capabilities already cited in this report. [36]


Start
Need stakeholder-grade living docs and requirements coverage?
Yes
No
Primary team stack is JVM/Java?
Top goal is modern browser automation and fast feedback?
Yes
No
Yes
No
Choose Serenity BDD
Use Cucumber only if Gherkin is strategic, otherwise reconsider Playwright
Choose Playwright
Is Gherkin a real cross-team contract?
Yes
No
Need Playwright-class browser automation too?
Choose Cucumber-only plus a deliberate engine
Choose Playwright
Yes
No
Use Serenity BDD with Playwright modules
Use Serenity with current JVM stack

Migration and introduction checklist

Introducing Playwright

Start with **one high-value browser workflow**, wire it with `@playwright/test`, and explicitly add the browser install step because Playwright no longer auto-downloads browsers when you install the `playwright` package alone. In CI, enable **traces on first retry**, store the HTML report as an artifact, and use the generated GitHub Actions workflow or equivalent CI pattern from the official docs. [37]

If your team currently says it “needs BDD,” pressure-test that assumption early. Official Playwright already supports **tags and annotations**, and many teams discover that code-first scenarios plus good naming/reporting are enough; adding Gherkin only pays off when business-readable scenarios are genuinely reviewed and maintained by more than QA. [38]

Introducing Serenity BDD

Decide the execution engine first: choose **Serenity + Selenium/WebDriver** if legacy compatibility and existing page objects dominate, or **Serenity + Playwright** if the real priority is modern browser automation on the JVM. Then standardize on **JUnit 5 / modern Cucumber setup** if you want the cleanest parallel path. [39]

Define your **requirements hierarchy**, tags, and report taxonomy early. Serenity’s biggest advantage is only realized when reports reflect product capabilities, not just test-class structure. If you are moving to current Serenity 5 with Cucumber/JUnit 5, note the documented **Cucumber plugin path change** for Serenity’s parallel reporter. [40]

Keeping or adopting Cucumber-only

Choose the missing pieces explicitly: **browser engine**, **API client**, **assertion library**, **state/DI approach**, **formatter/reporting destination**, and **parallel model**. Official docs explicitly say Cucumber does not ship browser or API automation itself, and Cucumber-JVM also does not come with an assertion library. [41]

If you stay with Cucumber-only, enforce two disciplines early: keep **scenario state isolated** and use **retries sparingly**. Cucumber docs recommend DI modules on the JVM to avoid shared static state, and Cucumber-JS retry support is best treated as a tactical exception for tagged flaky scenarios, not a default operating model. [42]

Primary sources and current community signals

The primary evidence in this report comes from the **official Serenity BDD docs and repo/release metadata**, the **official Playwright docs and GitHub/npm release metadata**, and the **official Cucumber docs plus Cucumber-JVM/Cucumber-JS repo and package data**. That source mix is the right one for this comparison because it captures both **documented capability** and **current maintenance cadence**. [43]

The community signal is directionally clear. On GitHub, Playwright's repo is far larger publicly than Serenity's, while Cucumber's community is split across multiple implementations. On Stack Overflow, the **Playwright** tag wiki is associated with related ecosystems across JavaScript, Python, TypeScript, and Java; **Cucumber** remains deeply tied to Java/Selenium/Ruby-era usage patterns; **Serenity-BDD** is visibly narrower and strongly concentrated around Java, Selenium, and Cucumber. [44]

Reddit is anecdotal rather than authoritative, but the current signal is still useful: recent **r/Playwright** and **r/softwaretesting** results show frequent discussion around Playwright career paths, CI observability, framework design, and day-to-day usage, while a current **r/softwaretesting** search result explicitly asks whether enterprise teams have moved away from **Gherkin** for acceptance tests. That does not prove decline, but it does suggest that **Playwright mindshare is growing faster than BDD consensus**. [45]

---

[1] [7] [24] [33] https://playwright.dev/docs/intro

https://playwright.dev/docs/intro

[2] [5] [10] [23] [26] [36] [43] https://serenity-bdd.github.io/docs/playwright/playwright_intro

https://serenity-bdd.github.io/docs/playwright/playwright_intro

[3] [41] https://cucumber.io/docs/guides/browser-automation/

https://cucumber.io/docs/guides/browser-automation/

[4] [11] [20] [29] [32] [40] https://serenity-bdd.github.io/docs/reporting/living_documentation

https://serenity-bdd.github.io/docs/reporting/living_documentation

[6] [18] https://github.com/serenity-bdd/serenity-core

https://github.com/serenity-bdd/serenity-core

[8] [25] [27] https://cucumber.io/docs/

https://cucumber.io/docs/

[9] [28] https://serenity-bdd.github.io/docs/tutorials/screenplay

https://serenity-bdd.github.io/docs/tutorials/screenplay

[12] [15] [34] [39] https://serenity-bdd.github.io/docs/guide/page_objects

https://serenity-bdd.github.io/docs/guide/page_objects

[13] https://serenity-bdd.github.io/docs/tutorials/rest

https://serenity-bdd.github.io/docs/tutorials/rest

[14] https://serenity-bdd.github.io/docs/tutorials/cicd-integration

https://serenity-bdd.github.io/docs/tutorials/cicd-integration

[16] https://serenity-bdd.github.io/docs/screenplay/screenplay_fundamentals

https://serenity-bdd.github.io/docs/screenplay/screenplay_fundamentals

[17] [22] https://mvnrepository.com/artifact/net.serenity-bdd/serenity-core

https://mvnrepository.com/artifact/net.serenity-bdd/serenity-core

[19]
https://serenity-bdd.github.io/docs/tutorials/running_cucumber_scenarios_in_parallel

https://serenity-bdd.github.io/docs/tutorials/running_cucumber_scenarios_in_parallel

[21] [44] https://github.com/microsoft/playwright

https://github.com/microsoft/playwright

[30] https://playwright.dev/docs/browsers

https://playwright.dev/docs/browsers

[31] https://playwright.dev/docs/test-parallel

https://playwright.dev/docs/test-parallel

[35] https://cucumber.io/docs/installation/

https://cucumber.io/docs/installation/

[37] https://playwright.dev/docs/release-notes

https://playwright.dev/docs/release-notes

[38] https://playwright.dev/docs/test-annotations

https://playwright.dev/docs/test-annotations

[42] https://cucumber.io/docs/cucumber/state/

https://cucumber.io/docs/cucumber/state/

[45] https://www.reddit.com/r/Playwright/new/

https://www.reddit.com/r/Playwright/new/